NutNet multidimensional stability analyses

04a: Check temporal effects on stability dimensionality

Qiang Yang

2026-06-22

Overview

This document provides a preliminary analysis of whether the dimensionality of ecological stability changes systematically with the duration of nutrient addition. In the main manuscript, the dimensionality analysis focuses on the effects of relative nutrient enrichment intensity. This preliminary workflow evaluates whether treatment duration itself needs to be included as a main driver of stability dimensionality.

Starting from the plot-level stability metrics generated in Step 2, the workflow calculates the dimensionality of ecological stability using the four stability components used in the manuscript: temporal functional variability, extent of functional change, temporal compositional variability, and extent of compositional change. For functional variaility, the analysis uses the detrended form of temporal functional variability.

The workflow first constructs observed dimensionality datasets for each nutrient treatment, temporal window, and enrichment-intensity group. It then generates null expectations by permuting enrichment-intensity assignments among plots and permuting the sequence of treatment years within plots while preserving the multivariate configuration of the stability components. Finally, simple regression models are fitted to the observed and permuted datasets to evaluate whether the observed temporal effects differ from the permutation-based null expectation.

This analysis is used as a supporting check. It shows that temporal duration has no detectable effect on stability dimensionality and supports the subsequent main dimensionality analysis, which focuses on nutrient enrichment intensity using generalized least-squares models with temporal autocorrelation in step 04b.

The complete workflow typically requires approximately 15 minutes to run on a standard desktop computer. To reduce computational time, the number of permutation iterations at line 161 can be decreased (e.g., from 1,000 to 100), although this may reduce the precision of the estimated null distributions and associated significance tests.

Output

Running this workflow generates the following outputs:

A figure showing permutation-based null distributions of regression coefficients and the corresponding observed coefficients for the effects of treatment duration, enrichment-intensity group, and their interaction (Fig. S8c).

Section 1: Set up the work environment

```
rm(list = ls())
gc()
```

```
##          used (Mb) gc trigger (Mb) max used (Mb)
```

```
## Ncells 496399 26.6    1075077 57.5   686411 36.7
## Vcells 927969  7.1    8388608 64.0  1876579 14.4
```

```r
# Load required R packages
Packages <- c(
  "tidyverse", "magrittr", "grid", "gridExtra", "scales",
  "broom", "dlookr", "nlme", "patchwork", "cowplot"
)
pacman::p_load(char = Packages)
# Explicitly use dplyr versions of common functions that may be masked by other packages
select <- dplyr::select
summarise <- dplyr::summarise
# Define input and output paths
main.path <- getwd()
data.path <- paste0(main.path, "/mydata")
figure.path <- paste0(main.path, "/myfigure")
set.seed(1234)
```

Section 2: Prepare observed dimensionality datasets

This section prepares the observed datasets used to calculate stability dimensionality. The analysis uses the detrended measure of temporal functional variability (`func_var_det`), together with the other three stability components used in the manuscript.

2.1 Organize the observed data

All stability components are standardized before dimensionality is calculated so that differences in measurement scale do not dominate the covariance structure.

```r
load(paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(paste0(data.path, "/df.tidy.Rdata"))
df.full <- df.tidy.stabilities %>%
  left_join(df.tidy %>% select(-contains("data")))
rm(df.tidy, df.tidy.stabilities)

df.full %<>%
  select(site_code:end_year, matches("_added_proportion|change|var"))
```

```
sta.names <- colnames(df.full) %>% str_subset("change|var")

# For the dimensionality analysis, all four stability components must be available for each row.
# Rows containing missing or infinite values are removed before covariance matrices are calculated.
df.full <- df.full %>%
  filter_all(all_vars(!is.infinite(.))) %>%
  filter_all(all_vars(!is.na(.)))

df.stability <- df.full

# Standardize stability components before calculating covariance matrices
df.stability %<>% mutate(across(one_of(sta.names), scale))


# Create metadata defining nutrient treatments and the final temporal window used for analysis
trt.set <- c("N", "P", "K")
final.year.set <- 9 # original using 5:9
df.meta <- crossing(trt.set, final.year.set)

## Add data nested by treatment and temporal window

df.meta <- df.meta %>%
  mutate(data = map2(
    trt.set, final.year.set,
    function(x1, x2) {
      plots.to.keep <- df.stability %>%
        filter(
          trt == x1,
          end_year == x2
        ) %>%
        select(site_code, block, trt, plot)
      subdat <- df.stability %>%
        semi_join(plots.to.keep) %>%
        filter(
          trt == x1,
          end_year <= x2
        )
      return(subdat)
```

```
    }
  ))
df.meta %<>% ungroup()

## Group plots by relative nutrient enrichment intensity and end year

df.meta <- df.meta %>%
  mutate(data2 = map2(
    trt.set, data,
    function(x1, dat) {
      df.pert <- dat %>%
        select(site_code, block, trt, plot, paste0(x1, "_added_proportion")) %>%
        distinct() %>%
        set_colnames(c("site_code", "block", "trt", "plot", "PI"))
      df.pert %<>%
        mutate(PI.group = ifelse(PI > median(PI), "High", "Low"))
      dat %<>% left_join(df.pert)
      dat.nested <- dat %>%
        group_by(end_year, PI.group) %>%
        nest() %>%
        ungroup()
      return(dat.nested)
    }
  ))

df.meta %<>%
  select(-data) %>%
  unnest(cols = data2)

df.N.P.K <- df.meta %>% ungroup()

# Retain only stability components for dimensionality calculations
df.N.P.K %<>%
  mutate(data = map(data, function(dat) dat %>% select(one_of(sta.names))))
df.N.P.K.obs <- df.N.P.K
```

2.2 Generate permutation datasets

This section generates permutation datasets for the preliminary temporal analysis. Enrichment-intensity assignments are randomized among plots within each nutrient treatment, and treatment years are randomized within plots. This preserves the multivariate structure of the stability components while breaking the observed association between dimensionality, enrichment intensity, and treatment duration.

```
set.seed(1234)
L <- list()
for (i in 1:1000) {
  # monitor the progress of the permutation procedure
  if(i %% 50 == 0) {
    print(paste("Permutation", i, "out of 1000"))
    print(Sys.time())
  }
  df.stability.obs <- df.stability
# First randomize relative nutrient enrichment intensity among plots within treatments
df.nutrient.pert <- df.stability.obs %>%
  select(site_code, block, trt, plot, contains("proportion")) %>%
  distinct()# the nutrient perturbation of each plot
df.nutrient.pert.random <- df.nutrient.pert %>%
  group_by(trt) %>%
  nest() %>%
  mutate(data2 = map(data, function(dt_){
    dt_ %>%
      mutate(N_added_proportion = sample(N_added_proportion),
             P_added_proportion = sample(P_added_proportion),
             K_added_proportion = sample(K_added_proportion))
  })) %>%
  select(-data) %>%
  unnest(cols = data2)
df.stability.random.plots <- df.stability.obs %>%
  select(-contains("proportion")) %>%
  left_join(df.nutrient.pert.random)

# Then randomize treatment years within each plot
# The multivariate configuration of the stability components is preserved.
# Only the treatment-year labels are randomized within plots.

# Plots with only one end-year cannot have their year sequence permuted.
```

```
# These plots are separated and added back after permuting the remaining plots.
df.stability.onlyone.year <- df.stability.random.plots %>%
  count(site_code, block, trt, plot) %>%
  filter(n == 1) %>%
  left_join(df.stability.random.plots) %>%
  select(-n)

df.stability.random.plots.AND.years <- df.stability.random.plots%>%
  anti_join(df.stability.onlyone.year) %>%
  group_by(site_code, block, trt, plot) %>%
  nest() %>%
  mutate(data = map(data, function(dt_){
    dt_ %>% mutate(end_year = sample(end_year))
  })) %>%
  unnest(cols = data)

df.stability.0 <- df.stability.onlyone.year %>% bind_rows(df.stability.random.plots.AND.years)

  # Create metadata defining nutrient treatments and the final temporal window used for analysis
  trt.set <- c("N", "P", "K")
  final.year.set <- 9 # original using 5:9
  df.meta <- crossing(trt.set, final.year.set)

  ## Add data nested by treatment and temporal window

  df.meta <- df.meta %>%
    mutate(data = map2(
      trt.set, final.year.set,
      function(x1, x2) {
        plots.to.keep <- df.stability.0 %>%
          filter(
            trt == x1,
            end_year == x2
          ) %>%
          select(site_code, block, trt, plot)
        subdat <- df.stability.0 %>%
          semi_join(plots.to.keep) %>%
          filter(
```

```
          trt == x1,
          end_year <= x2
        )
      return(subdat)
    }
  ))
df.meta %<>% ungroup()

## Group plots by relative nutrient enrichment intensity and end year

df.meta <- df.meta %>%
  mutate(data2 = map2(
    trt.set, data,
    function(x1, dat) {
      df.pert <- dat %>%
        select(site_code, block, trt, plot, paste0(x1, "_added_proportion")) %>%
        distinct() %>%
        set_colnames(c("site_code", "block", "trt", "plot", "PI"))
      df.pert %<>%
        mutate(PI.group = ifelse(PI > median(PI), "High", "Low"))
      dat %<>% left_join(df.pert)
      dat.nested <- dat %>%
        group_by(end_year, PI.group) %>%
        nest() %>%
        ungroup()
      return(dat.nested)
    }
  ))

df.meta %<>%
  select(-data) %>%
  unnest(cols = data2)

df.N.P.K <- df.meta %>% ungroup()

# Retain only stability components for dimensionality calculations
df.N.P.K %<>%
  mutate(data = map(data, function(dat) dat %>% select(one_of(sta.names))))
```

```
  L[[i]] <- df.N.P.K
  rm(df.N.P.K)
  rm(df.stability.0)
}
```

```
## [1] "Permutation 50 out of 1000"
## [1] "2026-06-22 12:23:26 CEST"

## [1] "Permutation 100 out of 1000"
## [1] "2026-06-22 12:24:07 CEST"

## [1] "Permutation 150 out of 1000"
## [1] "2026-06-22 12:24:52 CEST"

## [1] "Permutation 200 out of 1000"
## [1] "2026-06-22 12:25:41 CEST"

## [1] "Permutation 250 out of 1000"
## [1] "2026-06-22 12:26:25 CEST"

## [1] "Permutation 300 out of 1000"
## [1] "2026-06-22 12:27:06 CEST"

## [1] "Permutation 350 out of 1000"
## [1] "2026-06-22 12:27:46 CEST"

## [1] "Permutation 400 out of 1000"
## [1] "2026-06-22 12:28:30 CEST"

## [1] "Permutation 450 out of 1000"
## [1] "2026-06-22 12:29:13 CEST"

## [1] "Permutation 500 out of 1000"
## [1] "2026-06-22 12:29:58 CEST"
```

```
## [1] "Permutation 550 out of 1000"
## [1] "2026-06-22 12:30:41 CEST"

## [1] "Permutation 600 out of 1000"
## [1] "2026-06-22 12:31:25 CEST"

## [1] "Permutation 650 out of 1000"
## [1] "2026-06-22 12:32:11 CEST"

## [1] "Permutation 700 out of 1000"
## [1] "2026-06-22 12:32:57 CEST"

## [1] "Permutation 750 out of 1000"
## [1] "2026-06-22 12:33:39 CEST"

## [1] "Permutation 800 out of 1000"
## [1] "2026-06-22 12:34:21 CEST"

## [1] "Permutation 850 out of 1000"
## [1] "2026-06-22 12:35:13 CEST"

## [1] "Permutation 900 out of 1000"
## [1] "2026-06-22 12:36:01 CEST"

## [1] "Permutation 950 out of 1000"
## [1] "2026-06-22 12:36:49 CEST"

## [1] "Permutation 1000 out of 1000"
## [1] "2026-06-22 12:37:31 CEST"
```

```r
names(L) <- 1:length(L)
df.N.P.K.perm <- tibble(
  perm.id = names(L), # This will be the ID column
  data = L # This will be the column containing data.frames
) %>%
  unnest(data) %>%
```

```r
  mutate(perm.id = as.numeric(perm.id))

df.permutation <- df.N.P.K.obs %>%
  mutate(
    data.group = "obs",
    perm.id = 0
  ) %>%
  bind_rows(df.N.P.K.perm %>%
    mutate(data.group = "permutation"))
```

Section 3: Calculate stability dimensionality

This section calculates the dimensionality of ecological stability for both observed and permuted datasets. Dimensionality is quantified as the proportion of total multivariate variance explained by the first principal component of the four stability components. Higher values indicate stronger alignment among stability components and therefore lower dimensionality; lower values indicate weaker alignment and higher multidimensionality.

The non-detrended functional variability metric is removed here because the manuscript uses the detrended measure of temporal functional variability.

3.1 Calculate covariance matrices and dimensionality

```r
df.permutation <- df.permutation %>%
  mutate(data = map(data, function(dt1) dt1 %>% select(-func_var)))
# Calculate covariance matrices and eigenvectors for each nested dataset
df.permutation %<>% mutate(cov.M = map(data, cov))
df.permutation %<>% mutate(eigvec = map(cov.M, function(x) eigen(x)$vectors))
# Calculate the proportion of variance explained by the first principal component
df.permutation %<>%
  mutate(var.explained = map2_dbl(data, eigvec, function(D, X) {
    D <- as.matrix(D)
    P <- D %*% X
    V <- apply(P, 2, var)
    V[1] / sum(V)
  }))

df.variance.explained <- df.permutation %>%
  select(trt.set, final.year.set, end_year, PI.group, data.group, perm.id, var.explained)
```

Section 4: Test temporal effects using permutation-based regression

This section fits simple regression models to the observed and permuted dimensionality estimates. The response variable is stability dimensionality (`var.explained`), and the predictors are treatment duration (`end_year`), enrichment-intensity group (`PI.group`), and their interaction. The observed regression coefficients are compared with coefficients obtained from permutation datasets to assess whether temporal effects differ from the null expectation.

```
df.nested <- df.variance.explained %>%
  select(-final.year.set) %>% # unique value, so can be removed
  mutate(PI.group = as.factor(PI.group),
         PI.group = fct_relevel(PI.group, c("Low", "High"))) %>%
  group_by(trt.set, perm.id, data.group) %>%
  nest()
# Fit regression models for observed and permuted datasets
df.nested <- df.nested %>%
  mutate(model.set = map(data, function(.dt){
    .dt <- .dt %>% mutate(end_year = scale(end_year))
    lm(data=.dt, var.explained~end_year*PI.group)
  })) %>%
  mutate(coef.set = map(model.set, tidy)) %>%
  select(-model.set, -data) %>%
  unnest(cols = coef.set)


# Format nutrient treatment labels for plotting
df.nested <- df.nested %>%
  mutate(trt.set = paste0("+", trt.set)) %>%
  mutate(trt.set = as.factor(trt.set)) %>%
  mutate(trt.set = fct_relevel(trt.set, c("+N", "+P", "+K")))

# Organize regression outputs by nutrient treatment and model term
df.results <- df.nested %>%
  group_by(trt.set, term) %>%
  nest()
# Calculate permutation-based tail probabilities for observed coefficients
```

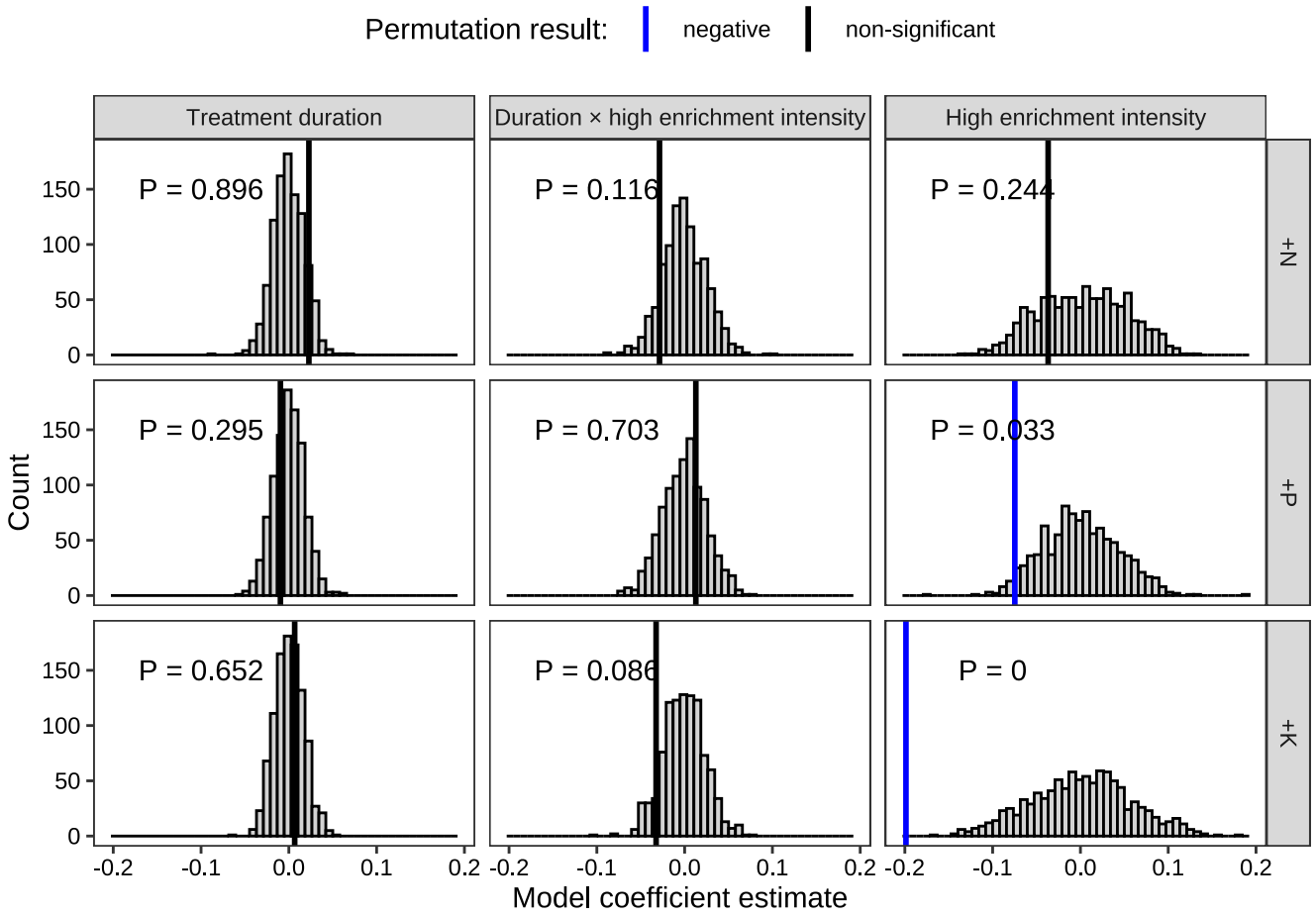
```r
df.results <- df.results %>%
  mutate(alpha = map_dbl(data, function(dt_) {
    x <- dt_ %>%
      filter(perm.id != 0) %>%
      select(estimate) %>%
      unlist()
    x0 <- dt_ %>%
      filter(perm.id == 0) %>%
      select(estimate) %>%
      unlist()
    length(which(x <= x0)) / length(x)
  }))
# Create text labels for permutation-based P values
df.results <- df.results %>%
  mutate(denote = map_chr(alpha, function(x) {
    paste("P =", x)
  }))
# Classify observed coefficients relative to the permutation distribution
df.results <- df.results %>%
  mutate(sig.group = map_chr(alpha, function(x) {
    if (x < 0.05) {
      y <- "negative"
    } else if (x < 0.95) {
      y <- "non-significant"
    } else {
      y <- "positive"
    }
  }))

# Visualize permutation distributions and observed regression coefficients
predictor.labs <- c("Treatment duration", "High enrichment intensity", "Duration × high enrichment intensity")
names(predictor.labs) <- c("end_year", "PI.groupHigh", "end_year:PI.groupHigh")

p.null <- df.nested %>%
  filter(data.group == "permutation", term!="(Intercept)") %>%
  ggplot(aes(x = estimate)) +
  facet_grid(trt.set~term,labeller = labeller(term = predictor.labs)) +
  geom_histogram(bins = 50, fill = "lightgray", color = "black") +
```

```
  geom_vline(
    data = df.nested %>%
      filter(data.group == "obs", term!="(Intercept)") %>%
      left_join(df.results %>% filter(term!="(Intercept)")%>% select(trt.set, sig.group)),
    aes(xintercept = estimate, color = sig.group),
    linewidth = 1
  ) +
  geom_text(
    data = df.results %>% filter(term!="(Intercept)") %>% mutate(x = -0.1, y = 150),
    aes(x = x, y = y, label = denote)
  ) +
  theme_bw() +
  xlab("Model coefficient estimate") +
  ylab("Count") +
  theme(
    axis.text = element_text(color = "black"),
    legend.position = "top",
    panel.grid = element_blank()
  ) +
  scale_color_manual(values = c("blue", "black")) +
  labs(color = "Permutation result:")

p.null
```


Permutation result:
negative
non-significant
Treatment duration
Duration × high enrichment intensity
High enrichment intensity
P = 0.896
P = 0.116
P = 0.244
P = 0.295
P = 0.703
P = 0.033
P = 0.652
P = 0.086
P = 0
+N
+P
+K
Count
Model coefficient estimate